Branch Prediction: Basics, Static Predictors

Dr Sparsh Mittal

Courtesy for some slides: Dr Onur Mutlu

An analogy

Consider a railroad junction:

 https://stackoverflow.com/questions/11227809/why-is-processing-a-sorted-array-faster-than-processing-an-unsorted-array

Direction prediction for the train

- You guess which direction the train will go!
- If you guessed right, it continues on.
- If you guessed wrong, the captain will stop, back up, and yell at you to flip the switch. Then it can restart down the other path.
- **If you guess right every time**, the train will never have to stop.
- **If you guess wrong too often**, the train will spend a lot of time stopping, backing up, and restarting.

What is a branch

A branch is a control-flow changing instruction.

It can be unconditional or conditional.

	Conditional branch	Unconditional branch (jump)

C language	if(x==5) {
	//Do something
	}
		goto

RISC-V ISA	bne x6, x8, loop	jal, jalr, j

The *for* loop involves conditional branch

- Consider the below for loop-code:

```
For (int j=0; j<N; j++) {
doSomething;
}
ExecuteInstructionAfterForLoop;
```

- Above for loop can be written as:

```
        j=0;
Loop: if(j>=N) goto Exit   ←——— Conditional branch
        doSomething;
        j++
        goto Loop
Exit:  ExecuteInstructionAfterForLoop;
```

Branch Prediction

- How do we keep the pipeline full in the presence of branches?
  - Guess the next instruction when a branch is fetched
  - Requires guessing the direction and target of a branch

- Branches can be unconditional or conditional

Jump/Branch Code	Unconditional/Conditional
j LOOP	Unconditional
beq x10 x0 Exit	Conditional
bgt, bge, ble, blt, bne	Conditional
jal x1 offset	Unconditional
jalr x1 x2 offset	Unconditional

- For conditional branches, we need to predict
  - **Whether the branch will be taken (=branch direction prediction)**
  - Branch target address (if taken) (=branch target address prediction)

  - Example
  - beq x10 x0 Exit #Assume Exit label is 52 bytes ahead of this instruction.
  - If(x10==x0) ➔ Condition is true. PC = PC+52
  - If(x10!=x0) ➔ Condition is false. PC = PC+4

  - Predicting whether the condition is true or false ➔ **direction prediction**
  - Predicting the address (or offset) to jump to ➔ target address prediction.

Definition: Backward and Forward Branch

- Backward branch: target address lower than branch PC
- Forward branch: target address higher than branch PC

```
LOOP:
   # ... some code to execute ...
   addi x11, x11, -1  #Decrement counter
   bne x11 x0 LOOP
```

Backward branch

```
li x11 10
LOOP: beq x11 x0 Exit
# ... some code to execute ...
addi x11, x11, -1  #Decrement counter
j LOOP
Exit: NOP
```

Forward branch

Solved example: Backward and Forward Branch

```
                ADDI  R1, R0, 0
          LOOP: MODI  R2, R1, 2
0xc44:          BNE   R2, R0, M2        // branch B1
                (do something A)
                ...
          M2:   MODI  R2, R1, 5
0xc84:          BNE   R2, R0, END       // branch B2
                (do something B)
                ...

          END:  ADDI  R1, R1, 1
0xcc0:          BNE   R1, 1000000, LOOP     // branch LP
```

- In above code, find whether B1, B2 and LP are backward or forward branches?
- Answer: B1 and B2 are forward. LP is backward

Definition: Biased Branch

- Biased branches: are either taken or not-taken most of the time

True most of the times

- For (i=0; i<100000; i++) {
- if(i==20000)

False most of the times

- //do something;
- if(i>10)

True most of the times

- //do something
- if(i%2 ==0)

Neither true, nor false for most of the times

- // do something
- }

All three are examples of biased branch

This is not a biased branch

Static and Dynamic instructions

- Static instruction count shows the number of instructions in a program
- Dynamic instruction count shows the actual number of instructions executed in a given program execution.
- Static and dynamic instruction counts can differ, for example,
  - due to branch operations, some instructions may not be executed
  - due to loops, some instructions may be executed multiple times

Solved Question

- If initially, x1 = 20. Find number of static and dynamic instruction executed by below code.
-
- loop: lb x2, 10(x1)
- addi x1, x1, -1
- bne x1, x0, loop
- Answer:
- Static: 3 and Dynamic: 60
- Explanation: this is a simple for loop, which executes for 20 times and each time, there are 3 instructions executed.

Solved example: Static and Dynamic instructions

```
loop:   ld    x2,   10(x1)
        add   x3,   x3,   x2
        sub   x1,   x1,   1
        bne   x1,   x0,   loop
```

- In above code, find number of static instructions
- Answer: 4
- If initially, x1 = 10. Find number of dynamic instructions
- Answer: 40

Some Direction Prediction Techniques

- Static Predictors (Compile time)
  - Always not taken
  - Always taken
  - BTFN (Backward taken, forward not taken)

- Dynamic Predictors (Run time)
  - One-level Predictor
    - One-bit Predictor (also called last time predictor)
    - Two-bit Predictor
  - Two-level predictor
    - Global History Predictor
    - Local History Predictor

STATIC BRANCH PREDICTION

Static Branch Prediction (I)

- Always not-taken
  - Simple to implement: no need for branch target buffer, no direction prediction
  - Low accuracy: ~30-40% (for conditional branches)

- Always taken
  - No direction prediction
  - Better accuracy: ~60-70% (for conditional branches)
    - Backward branches (i.e. loop branches) are usually taken

- Backward taken, forward not taken (BTFN)
  - Predict backward (loop) branches as taken, others not-taken

Solved Examples

- Consider this sequence: T, NT, NT, NT, T
- What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?
- Always-Taken Predictor:
- Actual T NT NT NT T
- Predict T T T T T
- Accuracy = 2/5 = 0.4
-
- Always-not-taken Predictor:
- Actual T NT NT NT T
- Predict NT NT NT NT NT
- Accuracy = 3/5 = 0.6

- Consider this sequence T, NT, T, NT, T
- What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?
-
- Always-Taken Predictor
- Actual T NT T NT T
- Predict T T T T T
- Accuracy = 3/5 = 0.6
-
- Always-not-taken Predictor
- Actual T NT T NT T
- Predict NT NT NT NT NT
- Accuracy = 2/5 = 0.4

Solved Question [1 of 3]

- Q. Consider this C code
-
- for (int i = 0; i < 1000000; i++) {
  if (i % 2 == 0) { // branch B1
  // do something A
  ...
  }
  if (i % 5 == 0) { // branch B2
  // do something B
  ...
  }
  }
-
  The corresponding assembly code is shown next

Solved Question [2 of 3]

```
                ADI  R1, R0, 0
        LOOP:   MODI R2, R1, 2
0xc44:          BNE  R2, R0, M2        // branch B1
                (do something A)
                ...
        M2:     MODI R2, R1, 5
0xc84:          BNE  R2, R0, END       // branch B2
                (do something B)
                ...

        END:    ADI  R1, R1, 1
0xcc0:          BNE  R1, 1000000, LOOP     // branch LP
```

- LP means the branch controlling the loop.
- The MODI (modulo-immediate) instruction is defined as follows [not for RISC-V]:
  MODI rd, rs, imm // rd <- rs mod imm

Solved Question [3 of 3]

- Q1. In steady state, how often are each of the three branches taken in the above code
- B1: ½. B2: 4/5 LP:1
- Branch1 checks odd/even condition, hence 50% taken-rate. Branch2 checks divisible by 5, hence 4/5 taken-rate. Branch LP is taken every-time, except the last time, when we exit the for loop.

- Q2. We design a static branch predictor. The static predictor predicts not taken for all forward branches, and taken for all backwards branches.
- For each branch, find accuracy of this static predictor?
- B1 and B2 are forward branches, and LP is a backward branch.
- B1: ½ B2: 1/5 LP: 1